PROGRAMMER'S REFERENCE GUIDE

Python
from first principles

A concise, comprehensive reference for developers already fluent in another language. Covers syntax, idioms, and the patterns that make Python distinctive.

01 — Basics | 02 — Types | 03 — Control Flow | 04 — Functions | 05 — OOP | 06 — Collections | 07 — Comprehensions | 08 — Exceptions | 09 — Files & I/O | 10 — Modules | 11 — Advanced | 12 — Stdlib

01 Basics & Syntax

Python uses **indentation** instead of braces. There are no semicolons. Blocks are defined purely by consistent whitespace (4 spaces is standard).

> Python is **dynamically typed** but strongly typed — implicit coercions don't happen. `"1" + 1` raises `TypeError`.

HELLO WORLD & VARIABLES

PYTHON

```python
# Variables — no declaration keyword needed
name = "Alice"
age = 30
pi = 3.14159
is_active = True          # bool literals are capitalised
nothing = None             # Python's null

# Multiple assignment
x, y, z = 1, 2, 3
a = b = c = 0              # chain assignment

# f-strings (Python 3.6+) — the preferred way to format
print(f"Hello, {name}! You are {age} years old.")
print(f"Pi is approximately {pi:.2f}")   # format spec in f-string
```

OPERATORS

```python
# Arithmetic
10 + 3   # 13
10 - 3   # 7
10 * 3   # 30
10 / 3   # 3.333... (always float)
10 // 3  # 3 (floor division)
10 % 3   # 1 (modulo)
2 ** 8   # 256 (exponentiation)
```

```python
# Logic & Comparison
x and y    # not &&
x or y     # not ||
not x      # not !

a == b     # equality
a != b     # inequality
a is b     # identity (same object)
a in lst   # membership test
```

> Use `==` for value equality and `is` for identity. Only use `is` with `None`, `True`, or `False`: `if x is None:`

STRING BASICS

```python
# Three quoting styles — all equivalent
s1 = 'single'
s2 = "double"
s3 = """triple-quoted
can span multiple lines"""

# Raw strings — backslashes not interpreted
path = r"C:\Users\Alice\Documents"

# Strings are immutable sequences — slice them like lists
s = "Hello, World!"
s[0]      # 'H'
s[-1]     # '!'
s[7:12]   # 'World'
s[::-1]   # '!dlroW ,olleH'  (reverse)

# Useful string methods
" hello ".strip()          # 'hello'
"hello".upper()           # 'HELLO'
"a,b,c".split(",")        # ['a', 'b', 'c']
",".join(["a", "b", "c"])  # 'a,b,c'
"hello".replace("l", "r") # 'herro'
```

02 Built-in Types

TYPE SYSTEM OVERVIEW

NUMERIC	SEQUENCE	MAPPING / SET
`int` `float` `complex` `bool`	`str` `list` `tuple` `range`	`dict` `set` `frozenset` `bytes`

```python
# Type introspection
type(42)          # <class 'int'>
isinstance(42, int) # True — preferred over type()

# Explicit type conversion
int("42")         # 42
float(5)          # 5.0
str(100)          # '100'
bool(0)           # False
list((1, 2, 3))   # [1, 2, 3]
```

TYPE HINTS (PYTHON 3.5+)

Python is still dynamically typed — hints are documentation and tooling aids, not runtime constraints.

```python
def greet(name: str) -> str:
    return f"Hello, {name}"

age: int = 30
scores: list[int] = [90, 85, 92]

from typing import Optional, Union

def find(key: str) -> Optional[int]:   # can return int or None
    ...
```

TRUTHINESS

Everything is truthy except: `None`, `False`, zero numbers, empty sequences/mappings.

```python
if my_list:           # True if non-empty — idiomatic Python
    ...
if value is not None: # explicit None check — preferred
    ...
```

03 Control Flow

IF / ELIF / ELSE

```python
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

# Ternary (conditional expression)
label = "even" if x % 2 == 0 else "odd"

# match/case (Python 3.10+) — structural pattern matching
match command:
    case "quit":
        quit()
    case "go" | "run":
        move()
    case _:              # wildcard — default
        print("unknown")
```

FOR LOOPS

```python
# Iterate directly over any iterable
for item in ["a", "b", "c"]:
    print(item)

# Range-based (like C for-loop)
for i in range(10):         # 0-9
    ...
for i in range(2, 10, 2):   # 2,4,6,8 (start, stop, step)
    ...

# With index — use enumerate(), not range(len())
for i, val in enumerate(["a", "b", "c"]):
    print(i, val)

# Unpack while iterating
pairs = [(1, "one"), (2, "two")]
for num, word in pairs:
    print(num, word)

# for...else — else runs if loop completed without break
for n in items:
    if n == target:
        break
else:
    print("not found")
```

WHILE LOOPS

```python
while condition:
    ...
    break      # exit loop
    continue   # skip to next iteration
    pass       # no-op placeholder
```

04 Functions

Functions are first-class objects — they can be passed, returned, and stored.

DEFINING FUNCTIONS

```python
def add(a, b):
    return a + b

# Default arguments
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

greet("Alice")              # positional
greet(greeting="Hi", name="Bob")  # keyword arguments
```

*ARGS AND **KWARGS

```python
# *args — variadic positional arguments (tuple)
def total(*nums):
    return sum(nums)

total(1, 2, 3, 4)   # 10

# **kwargs — variadic keyword arguments (dict)
def configure(**options):
    for key, val in options.items():
        print(f"{key} = {val}")

configure(debug=True, port=8080)

# Combined — order matters: positional, *args, kwonly, **kwargs
def func(pos, *args, kwonly=False, **kwargs):
    ...

# Unpacking at call site
args = [1, 2]
opts = {"verbose": True}
func(*args, **opts)
```

LAMBDA & HIGHER-ORDER FUNCTIONS

```python
# lambda: anonymous single-expression function
square = lambda x: x ** 2
square(5)   # 25

# map / filter / sorted with lambdas or functions
nums = [1, 2, 3, 4, 5]
evens = list(filter(lambda x: x % 2 == 0, nums))
doubled = list(map(lambda x: x * 2, nums))

people = [{"name": "Bob", "age": 30}, {"name": "Alice", "age": 25}]
sorted(people, key=lambda p: p["age"])
```

CLOSURES & DECORATORS

```python
# Closures
def make_multiplier(n):
    def multiply(x):
        return x * n      # n is captured from outer scope
    return multiply

triple = make_multiplier(3)
triple(5)   # 15

# Decorators — wrap a function to extend behaviour
import functools

def log_calls(func):
    @functools.wraps(func)        # preserves metadata
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__}")
        result = func(*args, **kwargs)
        print(f"Done")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

add(2, 3)   # prints "Calling add", then "Done"
```

GENERATORS

```python
# yield turns a function into a lazy generator
def count_up(n):
    for i in range(n):
        yield i           # suspends here, resumes on next()

for x in count_up(5):
    print(x)

# Generator expression (lazy version of list comprehension)
gen = (x**2 for x in range(1000000))  # no memory until iterated
```

05 Object-Oriented Programming

CLASSES

```python
class Animal:
    # Class variable — shared by all instances
    kingdom = "Animalia"

    def __init__(self, name: str, sound: str):
        # Instance variables — unique per instance
        self.name = name
        self._sound = sound     # _ = "private by convention"
        self.__secret = 42     # __ = name-mangled

    def speak(self):
        return f"{self.name} says {self._sound}"

    def __repr__(self):    # developer repr
        return f"Animal(name={self.name!r})"

    def __str__(self):     # user-friendly str()
        return self.name

    @classmethod
    def create_dog(cls):    # cls = the class itself
        return cls("Dog", "Woof")

    @staticmethod
    def describe():
        return "Animals are multicellular organisms"

cat = Animal("Cat", "Meow")
print(cat.speak())       # Cat says Meow
```

INHERITANCE

```python
class Dog(Animal):          # inherits from Animal
    def __init__(self, name, breed):
        super().__init__(name, "Woof")  # call parent __init__
        self.breed = breed

    def speak(self):           # override
        return f"{super().speak()}! *wags tail*"

# Multiple inheritance
class C(A, B):
    ...                         # MRO resolved via C3 linearization
```

PROPERTIES & DATACLASSES

```python
# @property — computed attributes, getters/setters
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value < 0: raise ValueError("Radius must be non-negative")
        self._radius = value

    @property
    def area(self):
        import math
        return math.pi * self._radius ** 2

# Dataclasses — auto-generate __init__, __repr__, __eq__
from dataclasses import dataclass, field

@dataclass
class Point:
    x: float
    y: float
    label: str = "origin"
    tags: list = field(default_factory=list)

p = Point(1.0, 2.0)
```

06 Collections

LISTS

```python
lst = [1, 2, 3, 4]

lst.append(5)          # [1,2,3,4,5]
lst.extend([6,7])       # [1,2,3,4,5,6,7]
lst.insert(0, 99)       # [99,1,2,3,4,5,6,7]
lst.pop()               # removes & returns last
lst.pop(0)              # removes & returns index 0
lst.remove(3)           # removes first occurrence of 3
lst.sort()              # in-place sort
sorted(lst)             # returns new sorted list
lst.reverse()           # in-place reverse
lst.index(5)            # index of first 5
len(lst)                # length
```

TUPLES

Like lists but **immutable**. Use for records, coordinates, return values.

```python
point = (3, 4)
x, y = point              # tuple unpacking
first, *rest = (1,2,3,4)  # first=1, rest=[2,3,4]

# Named tuple — like a lightweight struct
from collections import namedtuple
Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)
p.x, p.y    # 3, 4
```

DICTIONARIES

```python
d = {"name": "Alice", "age": 30}

d["name"]              # 'Alice' — KeyError if missing
d.get("name")          # 'Alice' — None if missing
d.get("city", "N/A")   # 'N/A' — default if missing
d["email"] = "a@b.com" # set
del d["age"]           # delete
"name" in d            # True — key membership

d.keys()               # dict_keys view
d.values()             # dict_values view
d.items()              # dict_items view — for key, val in d.items()

# Dict unpacking & merging (Python 3.9+)
merged = {**dict1, **dict2}
merged = dict1 | dict2   # Python 3.9+

# defaultdict — auto-creates missing keys
from collections import defaultdict
dd = defaultdict(list)
dd["group"].append(1)  # no KeyError
```

SETS

```python
s = {1, 2, 3}
s.add(4)
s.discard(2)     # no error if missing
s.remove(2)      # KeyError if missing

a | b   # union
a & b   # intersection
a - b   # difference
a ^ b   # symmetric difference
a <= b  # subset check
```

07 Comprehensions

Comprehensions are a Pythonic way to build collections in one expressive line.

```python
# List comprehension: [expr for item in iterable if condition]
squares = [x**2 for x in range(10)]
evens   = [x for x in range(20) if x % 2 == 0]

# Nested — flatten a 2D list
flat = [x for row in matrix for x in row]

# Dict comprehension
word_lengths = {word: len(word) for word in words}
inverted = {v: k for k, v in d.items()}

# Set comprehension
unique_lengths = {len(w) for w in words}

# Generator expression — lazy, no [] overhead
total = sum(x**2 for x in range(1_000_000))
```

> Prefer a comprehension over `map()`/`filter()` — they're more readable. Use a generator expression when you only need to iterate once and don't need to store the result.

08 Exception Handling

```python
try:
    result = 10 / x
except ZeroDivisionError:
    print("Cannot divide by zero")
except (TypeError, ValueError) as e:
    print(f"Bad input: {e}")
except Exception as e:
    print(f"Unexpected: {e}")
    raise               # re-raise original exception
else:
    print("No exception occurred")   # runs only on success
finally:
    print("Always runs — cleanup here")

# Raising exceptions
raise ValueError("Must be positive")

# Custom exceptions
class AppError(Exception):
    def __init__(self, message, code=None):
        super().__init__(message)
        self.code = code

raise AppError("Not found", code=404)
```

COMMON EXCEPTION TYPES

EXCEPTION	WHEN IT OCCURS
`ValueError`	Right type, wrong value (e.g. int("abc"))
`TypeError`	Wrong type for operation
`KeyError`	Dict key not found
`IndexError`	List index out of range
`AttributeError`	Object has no such attribute
`FileNotFoundError`	File/directory doesn't exist
`ImportError`	Module not found
`StopIteration`	Iterator exhausted

09 Files & I/O

```python
# Always use context manager — auto-closes the file
with open("file.txt", "r") as f:
    content = f.read()          # whole file as string

with open("file.txt") as f:
    lines = f.readlines()       # list of lines

with open("file.txt") as f:
    for line in f:               # lazy line iteration (memory-efficient)
        print(line.strip())

# Write modes: "w" (overwrite), "a" (append), "x" (exclusive create)
with open("out.txt", "w") as f:
    f.write("Hello\n")
    f.writelines(["line1\n", "line2\n"])

# Binary mode — add 'b': "rb", "wb"
with open("image.png", "rb") as f:
    data = f.read()

# Encoding — always specify for text files
with open("file.txt", encoding="utf-8") as f:
    ...
```

PATHLIB (PREFERRED OVER OS.PATH)

```python
from pathlib import Path

p = Path("/home/user/docs")
p / "file.txt"        # /home/user/docs/file.txt  (join with /)
p.exists()           # bool
p.is_file()
p.is_dir()
p.stem               # filename without extension
p.suffix             # '.txt'
p.parent             # parent directory Path
p.mkdir(parents=True, exist_ok=True)
list(p.glob("*.txt"))  # all .txt files
p.read_text()         # shorthand for open().read()
p.write_text("hello")
```

JSON & CSV

```python
import json

# Serialize to JSON
text = json.dumps({"name": "Alice", "age": 30}, indent=2)

# Parse from JSON
data = json.loads(text)

# File I/O
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)

import csv

with open("data.csv") as f:
    reader = csv.DictReader(f)     # rows as dicts using header
    rows = list(reader)
```

10

Modules & Packages

```python
# Import styles
import math
import math as m
from math import sqrt, pi
from math import *         # avoid — pollutes namespace

math.sqrt(16)
m.pi
sqrt(16)
```

CREATING A MODULE / PACKAGE

```python
# mypackage/
#   __init__.py      — makes directory a package
#   utils.py         — module
#   models/
#     __init__.py
#     user.py

from mypackage.utils import helper
from mypackage.models.user import User

# __init__.py can expose public API
# mypackage/__init__.py:
from .utils import helper     # relative import
```

VIRTUAL ENVIRONMENTS & PIP

```
# Create a virtual environment
python -m venv .venv
source .venv/bin/activate      # Linux/macOS
.venv\Scripts\activate         # Windows

# Install packages
pip install requests
pip install "fastapi[all]"==0.104.0
pip freeze > requirements.txt
pip install -r requirements.txt
```

GUARD FOR SCRIPT ENTRY POINT

```python
def main():
    print("Running as script")

if __name__ == "__main__":   # only runs when executed directly
    main()
```

11

Advanced Features

CONTEXT MANAGERS

```python
# Implement __enter__ / __exit__
class Timer:
    def __enter__(self):
        import time
        self.start = time.perf_counter()
        return self

    def __exit__(self, *args):
        import time
        print(f"Elapsed: {time.perf_counter() - self.start:.3f}s")

with Timer() as t:
    # timed code
    ...

# Or use contextlib.contextmanager decorator
from contextlib import contextmanager

@contextmanager
def managed_resource():
    resource = acquire()
    try:
        yield resource
    finally:
        resource.release()
```

ITERTOOLS & FUNCTOOLS

```python
from itertools import chain, islice, product, groupby, accumulate
from functools import reduce, partial, lru_cache, cache

# Memoization — cache function results
@lru_cache(maxsize=128)
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

# Partial application
double = partial(multiply, factor=2)

# Combine iterables without copying
for item in chain(list1, list2, list3):
    ...

# Take first N items from any iterable
first10 = list(islice(big_generator, 10))
```

ASYNC / AWAIT (ASYNCIO)

```python
import asyncio

async def fetch_data(url):
    await asyncio.sleep(1)    # simulate I/O
    return {"url": url, "data": "..."}

async def main():
    # Run sequentially
    result = await fetch_data("https://example.com")

    # Run concurrently
    results = await asyncio.gather(
        fetch_data("https://a.com"),
        fetch_data("https://b.com"),
    )

asyncio.run(main())    # entry point
```

DUNDER METHODS (MAGIC METHODS)

METHOD	TRIGGERED BY
`__init__`	Object creation
`__str__ / __repr__`	str(obj) / repr(obj)
`__len__`	len(obj)
`__getitem__`	obj[key]
`__setitem__`	obj[key] = val
`__contains__`	x in obj
`__iter__ / __next__`	for x in obj
`__eq__ / __lt__`	== / <
`__add__ / __mul__`	+ / *
`__enter__ / __exit__`	with statement
`__call__`	obj()

12

Essential Standard Library

DATA & MATH

`math` `decimal` `fractions` `statistics` `random` `collections` `heapq` `bisect`

SYSTEM & FILES

`os` `sys` `pathlib` `shutil` `tempfile` `glob` `argparse` `subprocess`

TIME & TEXT

`datetime` `time` `re` `string` `textwrap` `unicodedata`

NETWORKING & DATA

`json` `csv` `xml` `sqlite3` `http.client` `urllib` `socket` `ssl`

QUICK EXAMPLES

```python
# datetime
from datetime import datetime, timedelta
now = datetime.now()
tomorrow = now + timedelta(days=1)
print(now.strftime("%Y-%m-%d %H:%M"))

# re — regular expressions
import re
m = re.search(r"\d+", "abc 123 def")
m.group()        # '123'
re.findall(r"\b\w+\b", text)

# collections.Counter
from collections import Counter
c = Counter("abracadabra")
c.most_common(3)   # [('a',5), ('b',2), ('r',2)]

# os / sys
import os, sys
os.getcwd()
os.environ.get("HOME")
sys.argv         # command-line arguments
sys.exit(0)

# random
import random
random.choice(["a", "b", "c"])
random.randint(1, 100)
random.shuffle(my_list)
```

THIRD-PARTY ECOSYSTEM

WEB & APIS

`requests` `httpx` `fastapi` `flask` `django` `aiohttp`

DATA SCIENCE & ML

`numpy` `pandas` `matplotlib` `scikit-learn` `torch` `tensorflow`

> **PEP 8** — Python's style guide. Key rules: 4-space indent, 79-char line limit, snake_case for variables/functions, PascalCase for classes, UPPER_CASE for constants. Run `black` or `ruff` to auto-format.